Actividad Práctico-Experimental
Nro. 005

1. Datos Generales

Asignatura	Estructura de Datos
Ciclo	3ro A / B
Unidad	1: Estructuras de datos estáticas y dinámicas
Resultado de aprendizaje de la unidad	Listas Doblemente Enlazadas y Pilas: Navegación Bidireccional y Gestión LIFO de Mínimo Costo Energético
Práctica Nro.	005
Título de la Práctica	Listas Doblemente Enlazadas y Pilas: Navegación Bidireccional y Gestión LIFO de Mínimo Costo Energético
Nombre del Docente	Cristian Ramiro Narváez Guillén
Integrantes del Grupo 09	Isaac Vire Danny Beltran Axel Jimenez Lucas Orellana Noelia Bustan
Fecha	jueves 22 y viernes 23 de octubre 2026
Horario	Jueves 29 de octubre 10:30 – 11:30 Viernes 30 de octubre 11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

- Modear e implementar en **JavaScript (Node.js)** una estructura de **Lista Doblemente Enlazada** y una **Pila (Stack LIFO)** para la simulación de un módulo de navegación

bidireccional y un sistema transaccional de deshacer/rehacer (*Undo/Redo*) para el proyecto de gestión satelital sustentable.

- Evaluar experimentalmente la eficiencia temporal de inserción/eliminación en extremos frente a la reestructuración por desplazamiento en arreglos nativos.
- Cuantificar el consumo de memoria Heap generado por el almacenamiento de punteros dobles y mediante métricas del motor V8, garantizando el cumplimiento de criterios de Sostenibilidad Computacional (*Green Computing*).

3. Materiales y reactivos:

- Entorno de ejecución Node.js (versión LTS 20.x o superior) instalado.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para consultas a la documentación de estructuras dinámicas.
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).

4. Equipos y herramientas

Recursos Físicos

- Aula de Computación.

Computadora personal o de laboratorio:

- Sistema operativo: Windows, Linux o macOS.
- Procesador: Mínimo 2 núcleos.
- Memoria RAM: Mínimo 4 GB.

5. Procedimiento / Metodología

Enfoque PID-2025:

- **ABP (Aprendizaje Basado en Problemas):** Los estudiantes resolverán el reto de diseñar un subsistema de telemetría y auditoría de un satélite sustentable, el cual requiere almacenar eventos en un historial de navegación bidireccional (adelante/atrás) y ejecutar comandos de reversión inmediata de fallos en modo LIFO (*Last In, First Out*).
- **ABI (Aprendizaje Basado en Investigación):** Investigación empírica sobre la huella espacial de los punteros de 64 bits en la V8 Engine y la justificación matemática de la compensación entre espacio y tiempo (*Space-Time Trade-off*).

Tarea 1: Construcción de la Lista Doblemente Enlazada (Navegación Bidireccional)

1. En la carpeta del proyecto /APE_005_Listas_Pilas, cree el archivo ListaDoble.js.

2. Implemente la clase NodoDoble y la clase ListaDoblementeEnlazada garantizando que las referencias cruzadas se actualicen en tiempo constante O1.

JavaScript

```
/**
 * Representa un Nodo en una Lista Doblemente Enlazada.
 */
class NodoDoble {
  constructor(datos) {
        this.datos = datos; // Carga útil de telemetría satelital
        this.next = null;   // Referencia al siguiente nodo
        this.prev = null;   // Referencia al nodo anterior
  }
}

/**
 * Estructura de Datos Dinámica Doblemente Enlazada.
 */
class ListaDoblementeEnlazada {
  constructor() {
        this.head = null;
        this.tail = null;
        this.longitud = 0;
  }

  /**
   * Inserta un nuevo elemento al final de la lista.
   * Complejidad Temporal: O(1)
   */
  insertarFinal(datos) {
        const nuevoNodo = new NodoDoble(datos);
        if (!this.head) {
        this.head = nuevoNodo;
        this.tail = nuevoNodo;
        } else {
        this.tail.next = nuevoNodo;
        nuevoNodo.prev = this.tail;
        this.tail = nuevoNodo;
        }
        this.longitud++;
        return nuevoNodo;
  }

  /**
   * Elimina el último elemento de la lista.
```

```
 * Complejidad Temporal: O(1)
 */
 eliminarFinal() {
        if (!this.tail) return null;
        const nodoEliminado = this.tail;
        if (this.head === this.tail) {
        this.head = null;
        this.tail = null;
        } else {
        this.tail = this.tail.prev;
        this.tail.next = null;
        }
        nodoEliminado.prev = null;
        this.longitud--;
        return nodoEliminado.datos;
 }
}

module.exports = { NodoDoble, ListaDoblementeEnlazada };
```

Tarea 2: Implementación de la Pila Transaccional (Stack LIFO) sobre Punteros

1. Cree el archivo PilaTransaccional.js.
2. Desarrolle la Pila de comandos satelitales apoyada en nodos enlazados para evitar la relocalización de memoria de los arreglos dinámicos al hacer push o pop.

JavaScript

```
const { ListaDoblementeEnlazada } = require('./ListaDoble');

/**
 * Pila (Stack) orientada a Sostenibilidad Computacional.
 * Operaciones fundamentales Push y Pop en O(1) estricto.
 */
class PilaTransaccional {
 constructor() {
        this.lista = new ListaDoblementeEnlazada();
 }

 /**
  * Apila un comando de telemetría en el tope.
  * Complejidad Temporal: O(1)
  */
 push(comando) {
        this.lista.insertarFinal(comando);
 }
```

```
 /**
  * Desapila y retorna el comando en el tope (Undo).
  * Complejidad Temporal: O(1)
  */
 pop() {
        return this.lista.eliminarFinal();
 }

 /**
  * Retorna la cantidad de elementos almacenados.
  */
 obtenerTamano() {
        return this.lista.longitud;
 }
}

module.exports = PilaTransaccional;
```

Tarea 3: Experimento Comparativo (Benchmark CPU & Memory Heap)

1. Cree el archivo BenchmarkNavegacion.js.
2. Implemente un script que ejecute $N = 500.000$ inserciones y extracciones al inicio/final comparando un Array nativo de JavaScript frente a la PilaTransaccional.

JavaScript

```
const PilaTransaccional = require('./PilaTransaccional');

function obtenerMemoriaMB() {
  return (process.memoryUsage().heapUsed / 1024 / 1024).toFixed(2);
}

const N = 500000;
console.log(`=== BENCHMARK DE RENDIMIENTO (N = ${N}) ===\n`);

// 1. EVALUACIÓN CON ARREGLO TRADICIONAL (unshift / shift)
if (global.gc) global.gc();
const memInicialArray = process.memoryUsage().heapUsed;
console.time('Tiempo Array Nativo (unshift/shift)');

const arregloNativo = [];
for (let i = 0; i < N; i++) {
  arregloNativo.unshift({ id: i, log: 'Evento Satelital' }); // O(n)
}
for (let i = 0; i < N; i++) {
  arregloNativo.shift(); // O(n)
}
```

```
console.timeEnd('Tiempo Array Nativo (unshift/shift)');
const memFinalArray = process.memoryUsage().heapUsed;
const deltaMemArray = ((memFinalArray - memInicialArray) / 1024 / 1024).toFixed(2);
console.log(`Impacto de Memoria Heap Array: ${deltaMemArray} MB\n`);

// 2. EVALUACIÓN CON PILA DE PUNTEROS DINÁMICOS
if (global.gc) global.gc();
const memInicialPila = process.memoryUsage().heapUsed;
console.time('Tiempo Pila Transaccional O(1)');

const pilaSatelital = new PilaTransaccional();
for (let i = 0; i < N; i++) {
  pilaSatelital.push({ id: i, log: 'Evento Satelital' }); // O(1)
}
for (let i = 0; i < N; i++) {
  pilaSatelital.pop(); // O(1)
}

console.timeEnd('Tiempo Pila Transaccional O(1)');
const memFinalPila = process.memoryUsage().heapUsed;
const deltaMemPila = ((memFinalPila - memInicialPila) / 1024 / 1024).toFixed(2);
console.log(`Impacto de Memoria Heap Pila: ${deltaMemPila} MB\n`);
```

3. Ejecute el script con la bandera de recolección de basura:

Bash

```
node --expose-gc BenchmarkNavegacion.js
```

6. Resultados Esperados

Código de la Tarea 1

```javascript
/**
 * Representa un Nodo en una Lista Doblemente Enlazada.
 */
class NodoDoble {
  constructor(datos) {
    this.datos = datos; // Carga útil de telemetría satelital
    this.next = null;   // Referencia al siguiente nodo
    this.prev = null;   // Referencia al nodo anterior
  }
}

/**
 * Estructura de Datos Dinámica Doblemente Enlazada.
 */
class ListaDoblementeEnlazada {
  constructor() {
    this.head = null;
    this.tail = null;
    this.longitud = 0;
  }

  /**
   * Inserta un nuevo elemento al final de la lista.
   * Complejidad Temporal: O(1)
   */
  insertarFinal(datos) {
    const nuevoNodo = new NodoDoble(datos);
    if (!this.head) {
    this.head = nuevoNodo;
    this.tail = nuevoNodo;
    } else {
    this.tail.next = nuevoNodo;
    nuevoNodo.prev = this.tail;
    this.tail = nuevoNodo;
    }
    this.longitud++;
    return nuevoNodo;
  }

  /**
   * Elimina el último elemento de la lista.
   * Complejidad Temporal: O(1)
   */
  eliminarFinal() {
    if (!this.tail) return null;
    const nodoEliminado = this.tail;
    if (this.head === this.tail) {
    this.head = null;
    this.tail = null;
    } else {
    this.tail = this.tail.prev;
    this.tail.next = null;
    }
    nodoEliminado.prev = null;
    this.longitud--;
    return nodoEliminado.datos;
  }
}

module.exports = { NodoDoble, ListaDoblementeEnlazada };
```

Código de la Tarea 2

```
const { ListaDoblementeEnlazada } = require('./ListaDoble');

/**
 * Pila (Stack) orientada a Sostenibilidad Computacional.
 * Operaciones fundamentales Push y Pop en O(1) estricto.
 */
class PilaTransaccional {
  constructor() {
  this.lista = new ListaDoblementeEnlazada();
  }

  /**
   * Apila un comando de telemetría en el tope.
   * Complejidad Temporal: O(1)
   */
  push(comando) {
  this.lista.insertarFinal(comando);
  }

  /**
   * Desapila y retorna el comando en el tope (Undo).
   * Complejidad Temporal: O(1)
   */
  pop() {
  return this.lista.eliminarFinal();
  }

  /**
   * Retorna la cantidad de elementos almacenados.
   */
  obtenerTamano() {
  return this.lista.longitud;
  }
}
module.exports = PilaTransaccional;
```

Código de la Tarea 3

```
const PilaTransaccional = require('./PilaTransaccional');

function obtenerMemoriaMB() {
  return (process.memoryUsage().heapUsed / 1024 / 1024).toFixed(2);
}

const N = 500000;
console.log(`=== BENCHMARK DE RENDIMIENTO (N = ${N}) ===\n`);

// 1. EVALUACIÓN CON ARREGLO TRADICIONAL (unshift / shift)
if (global.gc) global.gc();
const memInicialArray = process.memoryUsage().heapUsed;
console.time('Tiempo Array Nativo (unshift/shift)');

const arregloNativo = [];
for (let i = 0; i < N; i++) {
  arregloNativo.unshift({ id: i, log: 'Evento Satelital' }); // O(n)
}
for (let i = 0; i < N; i++) {
  arregloNativo.shift(); // O(n)
}

console.timeEnd('Tiempo Array Nativo (unshift/shift)');
const memFinalArray = process.memoryUsage().heapUsed;
const deltaMemArray = ((memFinalArray - memInicialArray) / 1024 / 1024).toFixed(2);
console.log(`Impacto de Memoria Heap Array: ${deltaMemArray} MB\n`);
```

```
// 2. EVALUACIÓN CON PILA DE PUNTEROS DINÁMICOS
if (global.gc) global.gc();
const memInicialPila = process.memoryUsage().heapUsed;
console.time('Tiempo Pila Transaccional O(1)');

const pilaSatelital = new PilaTransaccional();
for (let i = 0; i < N; i++) {
  pilaSatelital.push({ id: i, log: 'Evento Satelital' }); // O(1)
}
for (let i = 0; i < N; i++) {
  pilaSatelital.pop(); // O(1)
}

console.timeEnd('Tiempo Pila Transaccional O(1)');
const memFinalPila = process.memoryUsage().heapUsed;
const deltaMemPila = ((memFinalPila - memInicialPila) / 1024 / 1024).toFixed(2);
console.log(`Impacto de Memoria Heap Pila: ${deltaMemPila} MB\n`);
```

Resultados de la ejecución de la Tarea 3

```
=== BENCHMARK DE RENDIMIENTO (N = 500000) ===

Tiempo Array Nativo (unshift/shift): 5:14.685 (m:ss.mmm)
Impacto de Memoria Heap Array: 26.50 MB

Tiempo Pila Transaccional O(1): 151.399ms
Impacto de Memoria Heap Pila: 42.52 MB
```

Análisis de los resultados

La forma en la que trabaja una pila y un array es el responsable del impacto tan diferente en el Heap y el tiempo de ejecución entre una implementación y otra. En el array se sacan y colocan información desde el inicio, lo que quiere decir que para eliminar o agregar información se tiene que reestructurar el array con cada operación, O(n), lo que ocupa más tiempo de ejecución, a cambio de menos almacenamiento ocupado en el Heap. Por otro lado, la pila, al trabajar con nodos, provoca que se ocupe más heap para guardar cada nodo y la referencia de este a sus nodos vecinos, pero a cambio, como no necesita reestructurarse toda la pila ante operaciones de inserción y eliminación, solo se necesitan cambiar pocas referencias, por lo que la operación de un O(1). La evidencia más fiel la podemos ver en los datos arrojados tras la ejecución 5 minutos en array vs 151 ms en pila, y 26.50 MB en array vs 42.52 MB en pila, dos estructuras con diferente complejidad de espacio y tiempo, se paga memoria para ahorrar tiempo de ejecución y trabajo de la CPU, lo que puede ser favorable en situaciones en las que se trabaja con grandes datos o se manipulan los extremos de manera frecuente. Claro que, se puede trabajar con push y pop para insertar y eliminar información en el array para hacer que este pueda tener mejores resultados en cuanto memoria.

7. Preguntas de Control:

Responda fundamentando sus argumentos bajo una perspectiva lógica, algorítmica y filosófica:

En el contexto de las estructuras de datos dinámicas, ¿cuál es la "sustancia" de un nodo en una Lista Doblemente Enlazada en comparación con la de una celda de un arreglo contiguo, y cómo modifica el puntero anterior (prev) su naturaleza constitutiva?

La sustancia de una celda en un arreglo contiguo es el valor de los datos y la dirección que a estos se les asigna, mediante un índice en un bloque de memoria de carácter secuencial. Por otro lado, la sustancia de un nodo en una lista doblemente enlazada es la unión de datos y datos de dirección, lo que permite que se pueda navegar por él sin mucho problema.

El puntero anterior, prev, modifica su naturaleza al aumentar su capacidad para enlazar y transformar la forma de navegación a uno bidireccional. Al ocurrir esto, solo se necesita la posición del nodo actual para poder operar de manera autónoma a cualquier nodo vecino cercano, por lo que la navegación, eliminación e inserción de datos se realizan de manera óptima en cualquier dirección.

¿Por qué la operación de inserción y extracción LIFO en una Pila implementada mediante punteros dobles alcanza la causa eficiente de la inmediatez O1, superando el trabajo mecánico de reubicación física de celdas que realiza el motor V8 en un arreglo nativo?

La operación de inserción y extracción LIFO en una pila con punteros dobles alcanza una eficiencia temporal de tiempo constante O(1) porque únicamente requiere actualizar un número fijo de enlaces lógicos entre elementos vecinos, es un proceso veloz e independiente del total de datos almacenados, a diferencia de esto los arreglos nativos del motor V8 organizan la información en posiciones contiguas de memoria, por lo que modificar o reestructurar su contenido suele exigir un trabajo mecánico de copiar, desplazar o reubicar físicamente sus celdas. Es por esto que la pila basada en punteros sustituye completamente la necesidad de mover datos en la memoria por una simple y directa reconexión de referencias.

¿Cuál es el fin último (*telos*) de la navegación bidireccional en un historial de operaciones o sistema de transacciones *Undo/Redo*, y en qué medida optimiza la estabilidad energética del sistema de cómputo?

La navegación bidireccional sirve principalmente para que el usuario pueda avanzar y retroceder entre sus acciones mediante las opciones de Deshacer (Undo) y Rehacer (Redo). Esto permite corregir errores y recuperar cambios de forma rápida, ordenada y segura, evitando tener que empezar el proceso desde cero.

En cuanto al consumo de energía, ayuda a optimizar los recursos del equipo porque evita repetir tareas innecesarias. Sin embargo, esto no asegura un ahorro energético automático, ya que guardar y recuperar el historial de cambios también consume memoria y procesamiento.

Aristóteles sostenía que nada existe sin una forma y materia dada. Si para ganar flexibilidad bidireccional se requiere duplicar las referencias de memoria prev y next, ¿cuándo deja el gasto de memoria Heap de ser una virtud de diseño para convertirse en un exceso ontológico (*overhead*)?

Lo empieza a ser en el momento que la flexibilidad otorgada se ve opacada por el gasto de RAM que provoca que un programa no pueda llevar a cabo sus ejecuciones de manera fluida y óptima. Desde otro punto de vista podríamos decir que la forma de una navegación bidireccional genera eficiencia que beneficia el programa, sin embargo, si la materia o la implementación misma de la navegación bidireccional es la a que a su vez hace que el programa pierda fluidez y consuma demasiados recursos, se esta pierde su propósito original, por tanto podemos decir que se convierte en un exceso ontológico.

Considerando la conservación de energía como principio técnico y ético, ¿de qué manera la eliminación instantánea de referencias sin desplazamiento de elementos mitiga el trabajo electromagnético de la CPU?

Lo que podemos ver es la eliminación instantánea, lo que hace básicamente que la CPU gaste menos energía por que al borrar un elemento de la lista solo se desconectan los punteros prev y next, lo cual es instantáneo. En cambio, si usamos un arreglo normal, el proceso tendría que mover todos los datos uno por uno y tapar el hueco que quedó. Al evitarnos todo ese trabajo pesado de reubicar cosas en la memoria, la CPU trabaja menos y ahorra energía.

8. Bibliografía

[1] B. Goetz, Java Concurrency in Practice. Addison-Wesley, 2022.

[2] P. W. Bible and L. Moser, An Open Guide to Data Structures and Algorithms. PALNI Open Press, 2023.

[3] J. Bloch, Effective Java, 3rd ed. Boston, MA: Addison-Wesley, 2018.

9. Anexos

Enlace al GitHub: https://github.com/B3kk4r/Estructura-de-datos-Grupo9.git